1. Import Libraries

In [22]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

2. Load Dataset

In [23]:
```
df = pd.read_csv('BostonHousing.csv')
df.head()
```

Out[23]:

	crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio	b	lsta
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90	4.9
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90	9.1
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.0
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.9
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.90	5.3

In [24]:
```
df.columns
#property
```

Out[24]:
```
Index(['crim', 'zn', 'indus', 'chas', 'nox', 'rm', 'age', 'dis', 'rad', 'tax',
       'ptratio', 'b', 'lstat', 'medv'],
      dtype='object')
```

In [25]:
```
# all independent but medv is dependent [medv value predict kraychi depending on

x= df[['crim', 'zn', 'indus', 'chas', 'nox', 'rm', 'age', 'dis', 'rad', 'tax',
       'ptratio', 'b', 'lstat']]
y=df['medv']
```

In [26]:
```
x
```

Out[26]:

	crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio	b	l
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90	4
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90	9
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.90	5
...	...	...	...	...	...	...	...	...	...	...	...	...	
501	0.06263	0.0	11.93	0	0.573	6.593	69.1	2.4786	1	273	21.0	391.99	9
502	0.04527	0.0	11.93	0	0.573	6.120	76.7	2.2875	1	273	21.0	396.90	9
503	0.06076	0.0	11.93	0	0.573	6.976	91.0	2.1675	1	273	21.0	396.90	5
504	0.10959	0.0	11.93	0	0.573	6.794	89.3	2.3889	1	273	21.0	393.45	6
505	0.04741	0.0	11.93	0	0.573	6.030	80.8	2.5050	1	273	21.0	396.90	7

506 rows × 13 columns

In [27]:
```
y
```

Out[27]:
```
0      24.0
1      21.6
2      34.7
3      33.4
4      36.2
       ...
501    22.4
502    20.6
503    23.9
504    22.0
505    11.9
Name: medv, Length: 506, dtype: float64
```

model train and test [main dataset split 80% train , 20% test]

In [28]:
```
#train_test_split() method return this 4 values
#test_size kiti madhe split kraych 0.20(20%)

x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42
```

Create Linear Regression Model

In [29]:
```
# find the relation between x_train and y and fit in model
model = LinearRegression()
model.fit(x_train, y_train)
```

Out[29]:

```
▾ LinearRegression
▶ Parameters
```

Predict Values

In [30]:

```
y_pred = model.predict(x_test)
```

In [31]:

```
y_pred
#jevdha testing data hota tyachi data sathi y value
```

Out[31]:

```
array([28.99672362, 36.02556534, 14.81694405, 25.03197915, 18.76987992,
       23.25442929, 17.66253818, 14.34119   , 23.01320703, 20.63245597,
       24.90850512, 18.63883645, -6.08842184, 21.75834668, 19.23922576,
       26.19319733, 20.64773313,  5.79472718, 40.50033966, 17.61289074,
       27.24909479, 30.06625441, 11.34179277, 24.16077616, 17.86058499,
       15.83609765, 22.78148106, 14.57704449, 22.43626052, 19.19631835,
       22.43383455, 25.21979081, 25.93909562, 17.70162434, 16.76911711,
       16.95125411, 31.23340153, 20.13246729, 23.76579011, 24.6322925 ,
       13.94204955, 32.25576301, 42.67251161, 17.32745046, 27.27618614,
       16.99310991, 14.07009109, 25.90341861, 20.29485982, 29.95339638,
       21.28860173, 34.34451856, 16.04739105, 26.22562412, 39.53939798,
       22.57950697, 18.84531367, 32.72531661, 25.0673037 , 12.88628956,
       22.68221908, 30.48287757, 31.52626806, 15.90148607, 20.22094826,
       16.71089812, 20.52384893, 25.96356264, 30.61607978, 11.59783023,
       20.51232627, 27.48111878, 11.01962332, 15.68096344, 23.79316251,
        6.19929359, 21.6039073 , 41.41377225, 18.76548695,  8.87931901,
       20.83076916, 13.25620627, 20.73963699,  9.36482222, 23.22444271,
       31.9155003 , 19.10228271, 25.51579303, 29.04256769, 20.14358566,
       25.5859787 ,  5.70159447, 20.09474756, 14.95069156, 12.50395648,
       20.72635294, 24.73957161, -0.164237  , 13.68486682, 16.18359697,
       22.27621999, 24.47902364])
```

In [32]:

```
#compare
model.score(x_train,y_train)
```

Out[32]:

```
0.7508856358979673
```

In [33]:

```
model.score(x_test,y_test)
#accuracy value
```

Out[33]:

```
0.6687594935356301
```

In [34]:

```
#find error [error kiti yet aahe konti aahe]
# mean_squared_error()   --> actual value and predict  madhe difference gheun ty
#mean error
np.sqrt(mean_squared_error(y_test,y_pred))
```

Out[34]:

```
np.float64(4.92860218266535)
```

In [41]:

```
# Scatter plot
plt.scatter(y_test, y_pred, color='green',marker='*')

#  Add perfect prediction line (diagonal line)
plt.plot([y_test.min(), y_test.max()],
         [y_test.min(), y_test.max()],
```

```
         color='green', linewidth=2)

# Labels
plt.xlabel("Actual Price")
plt.ylabel("Predicted Price")
plt.title("Actual vs Predicted House Prices")

# Show graph
plt.show()
```

In [44]:
```
plt.scatter(df['rm'], df['medv'])
plt.xlabel("Number of Rooms")
plt.ylabel("House Price")
plt.title("Rooms vs Price")
plt.show()
```

Rooms vs Price
House Price
Number of Rooms
50
40
30
20
10
4
5
6
7
8
9

In [ ]: